Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации

Ордена Трудового Красного Знамени

федеральное государственное бюджетное образовательное учреждение высшего образования

МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Математической кибернетики и информационных технологий»

Информационные технологии и программирование

Лабораторная работа №5

Выполнил:

студент группы БВТ-2306

Кесслер А. С.

Москва 2025

Цель работы

Ознакомиться с реализацией внедрения зависимостей (Dependency injection (DI)) в фреймворке Spring. Создать проект с помощью фреймворка Spring, в котором реализовать внедрение зависимости согласно варианту, осуществить его запуск.

Задание

13 Вариант

Базовый класс: Оружие

Дочерние классы: Меч, Лук, Волшебная палочка

Решение

Armory

```
package com.example;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
public class Armory {
    private final List<Weapon> weapons;

    @Autowired
    public Armory(List<Weapon> weapons) {
        this.weapons = weapons;
    }

    public void displayWeapons() {
        weapons.forEach(weapon -> {
            System.out.println("Оружие: " + weapon.getName());
            weapon.use();
        });
    }
}
```

Bow

```
package com.example;

import org.springframework.stereotype.Component;

@Component
```

```java
public class Bow implements Weapon {
    private String name;
    public Bow() {
        this.name = "Лук Робина";
    }
    @Override
    public String getName() { return name; }
    @Override
    public void use() {
        System.out.println(name + " выпускает стрелу!");
    }
}
```

MagicWand

```java
package com.example;

import org.springframework.stereotype.Component;

@Component
public class MagicWand implements Weapon {
    private String name;
    public MagicWand() {
        this.name = "Палочка Гарри Поттера";
    }
    @Override
    public String getName() { return name; }
    @Override
    public void use() {
        System.out.println(name + " создаёт магическую вспышку!");
    }
}
```

Main

```java
package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {
    public static void main(String[] args) {
        // ApplicationContext context = new
AnnotationConfigApplicationContext("com.example");
        // Armory armory = context.getBean(Armory.class);
        // armory.displayWeapons();

        // ApplicationContext context = new
AnnotationConfigApplicationContext(SpringConfig.class);
```

```
        // Armory armory = context.getBean(Armory.class);
        // armory.displayWeapons();

        ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
        Armory armory = context.getBean(Armory.class);
        armory.displayWeapons();
    }
}
```

Здесь у нас на выбор:

- Через аннотации
- Через xml-конфиг

SpringConfig

```
package com.example;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.List;

@Configuration
public class SpringConfig {
    @Bean
    public Weapon sword() {
        return new Sword();
    }
    @Bean
    public Weapon bow() {
        return new Bow();
    }
    @Bean
    public Weapon magicWand() {
        return new MagicWand();
    }
    @Bean
    public Armory armory() {
        return new Armory(List.of(sword(), bow(), magicWand()));
    }
}
```

Sword

```
package com.example;

import org.springframework.stereotype.Component;

@Component
```

```java
public class Sword implements Weapon {
    private String name;
    public Sword() {
        this.name = "Экскалибур";
    }
    @Override
    public String getName() { return name; }
    @Override
    public void use() {
        System.out.println(name + " наносит рубящий удар!");
    }
}
```

Weapon

```java
package com.example;

public interface Weapon {
    String getName();
    void use();
}
```

Pom

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>weapon-project</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-core</artifactId>
            <version>6.1.6</version>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>6.1.6</version>
        </dependency>
```

```
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.0.0</version>
                <configuration>
                    <mainClass>com.example.Main</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

ApplicationContext

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="sword" class="com.example.Sword"/>
    <bean id="bow" class="com.example.Bow"/>
    <bean id="magicWand" class="com.example.MagicWand"/>

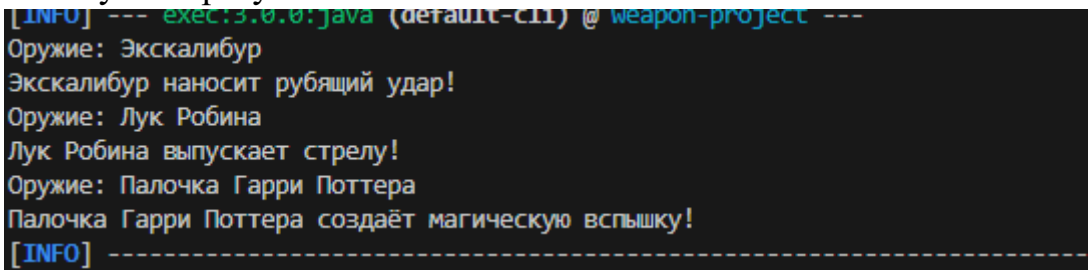
    <bean id="armory" class="com.example.Armory">
        <constructor-arg>
            <list>
                <ref bean="sword"/>
                <ref bean="bow"/>
                <ref bean="magicWand"/>
            </list>
        </constructor-arg>
    </bean>

</beans>
```

Запускаем код командами:

- mvn clean compile
- mvn exec:java

И получаем результат:



Вывод

В ходе работы мы освоили работу с Spring и научились внедрять зависимости разными методами.